

Технічний аналіз екосистеми DRAKON (Stepan Mitkin): Оцінка інтеграційного потенціалу

1. Executive Summary

Глибокий архітектурний та технічний аналіз екосистеми інструментів DRAKON, розробленої Степаном Міткіним, виявляє складну, високоінтегровану, але водночас технічно неоднорідну інфраструктуру. Дослідження підтверджує, що екосистема еволюціонувала від монолітних десктопних застосунків, орієнтованих на системне програмування, до хмарних платформ і вебвіджетів, призначених для візуального програмування та генерації коду.¹ Головною метою цього аналізу є визначення технічної придатності існуючих компонентів для інтеграції у сучасний вебзастосунок (Single Page Application) на базі штучного інтелекту, призначений для генерації та інтерактивного редагування алгоритмічних схем.

Аналіз кодових баз, розгорнутих у публічних репозиторіях, демонструє парадоксальну архітектурну парадигму: екосистема Міткіна є глибоко "самореферентною" (self-hosted). Більшість вебкомпонентів, серверної бізнес-логіки та генераторів коду створюються шляхом малювання візуальних діаграм у застарілому десктопному редакторі (DRAKON Editor), який компілює ці графічні моделі у мови JavaScript та Lua.³ Ця методологія, хоча й концептуально цікава, створює нездоланий технічний бар'єр для прямого повторного використання серверних компонентів (наприклад, бекенду DrakonHub) у рамках сучасних інженерних практик. Водночас дослідження виокремлює один критично важливий і технічно бездоганий компонент — клієнтську бібліотеку drakonwidget.js. Цей модуль є ізольованим, не має жодних зовнішніх залежностей і відповідає виключно за рендеринг та взаємодію з графом у браузері.⁵

✓ Перевірені факти вказують на те, що drakonwidget ідеально підходить для адаптації та обгортання у сучасні декларативні фреймворки (React, Vue, Svelte). Формат даних, який використовує віджет, являє собою плоский JSON-словник (adjacency list), що мінімізує структурну вкладеність.⁵ Ця топологічна особливість є стратегічною перевагою для генерації схем за допомогою великих мовних моделей (LLM), оскільки штучному інтелекту значно простіше генерувати та лінкувати плоскі масиви ідентифікаторів, ніж конструювати глибоко вкладені абстрактні синтаксичні дерева (AST).⁶ Отже, стратегія розробки AI-орієнтованого вебзастосунку має базуватися на повному відкиданні серверної інфраструктури Міткіна, розробці власного бекенду та глибокій інтеграції виключно фронтенд-віджета drakonwidget.js.

2. Ecosystem Map

Екосистема інструментів Степана Міткіна не є лінійним набором програм, а являє собою складну мережу взаємозалежних застосунків, які охоплюють декілька архітектурних шарів. Для розуміння інтеграційного потенціалу необхідно чітко класифікувати кожен шар та його функціональне призначення. Аналіз репозиторіїв та технічної документації дозволяє виділити чотири основні рівні абстракції.

Перший рівень — це **Desktop Layer (Фундаментальний рівень генерації та інструментарію)**. Ядром цього рівня і всієї екосистеми загалом виступає класичний `drakon_editor`.¹ Це десктопний застосунок, написаний мовою Tcl/Tk, що вимагає для своєї роботи середовища Tcl 8.6.² Його головна функція виходить далеко за межі простого малювання блок-схем. Він є повноцінним CASE-засобом (Computer-Aided Software Engineering) і кодогенератором.¹ Усі візуальні діаграми зберігаються у файлах з розширенням `.drn`, які насправді є повноцінними базами даних SQLite.¹ Редактор містить потужну підсистему плагінів, що дозволяє транслювати графічні алгоритми у вихідний код мовами C, C++, Java, C#, Python, Erlang, Lua, JavaScript та іншими.² ⚠ Важливий висновок на основі коду полягає в тому, що розробка нових інструментів (таких як DrakonHub) відбувається саме в цьому редакторі: логіка малюється у вигляді графа, а потім компілюється у цільову платформу.³ Для сучасного веброзробника цей інструмент має лише історичну та референсну цінність, оскільки його кодова база є архаїчною і не підлягає вебтрансформації.

Другий рівень — це **Web Rendering Layer (Рівень візуалізації та взаємодії)**. Тут розташовується репозиторій `drakonwidget`. Це чиста клієнтська JavaScript-бібліотека (vanilla JS), яка відповідає за рендеринг JSON-структур у Document Object Model (DOM) браузера або на елементі HTML5 Canvas.⁵ Вона є автономним модулем, що забезпечує як перегляд діаграм, так і складну взаємодію з ними (виділення, зміна розмірів, переміщення).⁵ ✅ Цей віджет є найбільш зрілим, стабільним і технічно ізольованим активом екосистеми. Відсутність залежностей (таких як jQuery, D3.js чи React) парадоксальним чином робить його ідеальним кандидатом для інтеграції, оскільки він не створює конфліктів версій і може бути обгорнутий у будь-який сучасний фронтенд-фреймворк.⁵

Третій рівень становить **Web Application Layer (Рівень хмарних застосунків)**, представлений проектом `drakonhub` та його сателітом `drakonhub_desktop`. DrakonHub — це SaaS-продукт для створення діаграм у браузері.³ Клієнтська частина цього продукту повністю покладається на `drakonwidget.js` для малювання.⁴ Однак його серверна частина викликає серйозні архітектурні застереження. Бекенд працює на базі Tarantool 2.x — платформи, яка поєднує в собі in-memory базу даних та сервер застосунків, що виконує Lua-скрипти.³ Аналіз репозиторію показує, що 73.9% коду — це JavaScript, а 15.9% — Lua, проте більшість цих файлів не пишуться розробниками вручну. Вони генеруються з візуальних моделей (файлів `.drn`) за допомогою десктопного DRAGON Editor.³ ⚠ Висновок на основі цієї архітектури очевидний: бекенд DrakonHub є абсолютно непідтримуваним (unmaintainable) для зовнішньої команди, оскільки для виправлення будь-якого серверного

багу необхідно редагувати Tcl/Tk моделі, компілювати їх у Lua, і розгортати в Tarantool. Також існує десктопна оболонка drakonhub_desktop, яка є простою веб-обгорткою (написаною на JS/HTML) для локальної роботи з діаграмами без доступу до інтернету.³

Четвертий рівень — це **IDE Layer (Рівень інтегрованих середовищ розробки)**, до якого належить drakon.tech та drakon.tech.desktop.¹ Це веборієнтовані середовища для візуального програмування. Вони реалізують концепцію «алгоритм як код» (DRAKON-2).¹ Графічні макроікони визначають потік управління (flow control), а всередині ікон пишуться фрагменти коду (наприклад, на JavaScript або Clojure).⁸ Платформа транслює ці візуальні структури в абстрактне синтаксичне дерево (AST), яке згодом компілюється в повноцінні виконувані модулі.¹ Нещодавно екосистема була розширена для підтримки генерації «псевдокоду» для систем штучного інтелекту (ChatGPT, Gemini, Grok), де візуальна логіка конвертується в оптимізовані текстові промпти.⁸ Хоча ці інструменти є функціонально потужними, їхній рушій кодогенерації міцно зчеплений з їхнім власним монолітним бекендом, що обмежує можливості прямого повторного використання у сторонніх мікросервісних проєктах.

3. Repository Inventory Table

Нижче наведено вичерпну технічну інвентаризацію ключових репозиторіїв екосистеми Степана Міткіна. Аналіз базується на метаданих кодових баз, датах останніх комітів (станом на 2026 рік) та структурі ліцензування. Оцінка "Integration Value" (Цінність для інтеграції) відображає придатність компонента для прямого використання у новому AI-орієнтованому проєкті без глибокого рефакторингу.

repo	призначення	стек	тип (editor/viewer/widget/parser)	input/output formats	останній коміт	ліцензія	integration value
drakon widget	Ядро рендерингу та управління станом візуального графа для	Vanilla JS (99.5%)	viewer / editor / widget	Input: JSON Output : JSON, Canvas DOM, BASE64 зображення ⁵	Активний (v1.4.7, 64 коміти) ⁵	Unlicense (еквівалент Public Domain) ⁵	Критично Висока. Повністю готовий фундамент для створе

	браузера. Забезпечує Манхеттенську маршрутизацію ліній без залежностей.						ння React/Vue обгортки. Не потребує переписування алгоритмів топології.
drakon.tech.desktop	Десктопна оболонка для генерації AI-промптів та вихідного коду (JavaScript, Clojure) з діаграм.	JS (96.6%), CSS (2.9%), HTML	desktop IDE / parser / prompt generator	Input: JSON (solution.json) Output: Псевдокод (промпти), foo-bar.cljc, .js файли ⁸	Активний (квітень 2026, 146 комітів) ⁸	Unlicense ¹⁰	Середня. Логіка генерації AST та псевдокоду є цінним референсом для навчання LLM розумінню структури DRAKON. Сама оболонка не потрібна.
drakon	Дескто	Tcl/Tk	desktop	Input:	Активн	Public	Низька

_editor	пний редактор-генератор, CASE-засіб. Ядро генерації коду екосистеми з .drn файлів.	(92.6%), SQLite, C (4.3%)	editor / code generator	.drn (SQLite бінарні файли) Output : C, Java, JS, Lua, Python, Erlang, Tcl ²	ий (179 комітів) ¹¹	Domain (з винятками для стороних шрифтів) ²	. Мова Tcl/Tk є застарілою для сучасного вебу. Корисний виключно для вивчення еталонної реалізації евристик компілятора графів.
drakon hub_desktop	Десктопна, офлайн-орієнтована версія хмарного редактора. Забезпечує локальне збереження файлів без	JS (89.8%), HTML (8.4%)	desktop wrapper / editor	Input: JSON Output : JSON, PNG, SVG ³	Активний (травень 2026, 65 комітів) ⁴	Public Domain ³	Низька . Логіка оболонки (shell) занадто тісно пов'язана зі специфікою UI Drakon Hub. Може бути корисним прикла

	синхро нізації з сервер ом.						дом ініціаліза ції віджета .
drakon hub	Повний вихідни й код сервер ної та клієнтс ької частин и SaaS-п родукт у (редакт ора діаграм та інтелек т-карт).	JS (73.9%), Lua (15.9%), Taranto ol 2.x	web applicat ion / server / storage	Input: HTTP REST, JSON Output : JSON	Активн ий (жовте нь 2024, 51 коміт) ³	Public Domain ³	Дуже низька. Архітек тура бекенд у на Taranto ol з автозге нерова ним з Tcl кодом Lua робить його непідтр имуван им (unmain tainable) для нових вебком анд. ³
drakon. tech	Повноц інна хмарна IDE для візуаль ного програ муванн я (перетв	JS (73.7%), Lua (14.1%), Emacs Lisp (8.9%)	web IDE / AST compile r	Input: JSON Output : Executa ble Code	Заморо жений (~2 роки тому, 133 коміти) ⁹	Unlicen se ⁹	Серед ня. Директ орія /genera tor містить цінні алгори тми

	орення алгори тмів на скінчен ні автома ти та код).						обходу графа, що трансл юють візуаль ну тополо гію в абстра ктні синтакс ичні дерева. 9
--	--	--	--	--	--	--	---

✓ **Стратегічний висновок щодо ліцензій:** Найбільшою нетехнічною перевагою екосистеми Степана Міткіна є її ліцензійна політика. Усі ключові компоненти випущені або у суспільне надбання (Public Domain), або під ліцензією Unlicense (яка є юридичним еквівалентом суспільного надбання).³ Це означає повну відсутність копірайт-ризиків, вимог щодо розкриття вихідного коду (як у GPL) або обов'язкової атрибуції. Будь-який компонент можна форкувати, модифікувати та закривати у комерційному пропрієтарному AI-продукті без жодних юридичних перешкод.

4. Data Formats Analysis

Ефективність AI-орієнтованого застосунку критично залежить від того, наскільки легко велика мовна модель (LLM, наприклад, GPT-4, Claude 3) здатна читати, розуміти та генерувати структури даних, що відповідають за візуалізацію. Аналіз екосистеми DRAGON виявляє конкуренцію двох форматів даних: бінарного (на базі SQLite) та текстового (на базі JSON).

4.1 Бінарний формат .drn (Legacy Storage)

Історично екосистема спиралася на файли .drn, які генерує drakon_editor.² ✓ **Перевірені факти:** Незважаючи на кастомне розширення, ці файли є повноцінними реляційними базами даних SQLite. Редактор використовує бібліотеку libsqlite3-tcl для взаємодії з ними.¹ ⚠ **Висновки на основі архітектури:** Хоча використання SQLite є потужним підходом для управління станом у десктопних застосунках (забезпечуючи транзакційність та цілісність величезних графів), для сучасного веб-SPA це створює непотрібний інженерний оверхед. Щоб парсити .drn у браузері, необхідно компілювати SQLite у WebAssembly (WASM) або будувати складні мости через бекенд. Крім того, LLM не здатні безпосередньо генерувати бінарні файли баз даних. Отже, від підтримки формату .drn у новому продукті слід

повністю відмовитися.

4.2 Текстовий формат JSON (drakonwidget Schema)

Для вебкомпонентів (drakonwidget, drakonhub, drakon.tech.desktop) використовується серіалізований формат JSON.⁵ Цей формат має унікальні характеристики, які роблять його ідеальним для взаємодії з алгоритмами машинного навчання та LLM.

Замість традиційного для багатьох бібліотек малювання (наприклад, React Flow) підходу з окремими масивами nodes (вузли) та edges (зв'язки), або глибоко вкладених дерев (Abstract Syntax Trees), drakonwidget використовує **плоский словник (adjacency list)**.

Згідно з офіційною документацією віджета⁵, структура JSON-представлення діаграми виглядає наступним чином:

Глобальний об'єкт (Diagram Object):

- name (string): Назва діаграми.
- access (string): Рівень доступу (наприклад, "read" або "write").
- params (string): Вхідні параметри функції алгоритму, що відображаються у спеціальному блоці.
- style (string): Серіалізований JSON-рядок, що містить глобальні перевизначення стилів.
- items (object): **Центральний елемент структури.** Це словник (Map/Dictionary), де ключами виступають унікальні ідентифікатори вузлів (наприклад, строкові числа "1", "14", "node_A"), а значеннями — об'єкти, що описують самі вузли (Item Objects).

? Розбіжності у документації: Варто зазначити, що у робочих документах проєкту Motia⁷ items (або nodes) іноді описується як масив (array). Однак, аналіз вихідної документації drakonwidget⁵ чітко вказує на те, що items — це словник (dictionary). Формат словника є значно ефективнішим ($O(1)$ для доступу по ID), і саме на нього слід орієнтуватися як на Canonical Data Model (CDM).

Структура внутрішнього представлення вузла (Item Object):

Кожен елемент словника items описує як візуальні, так і топологічні властивості ікони:

Поле об'єкта	Тип даних	Топологічне та функціональне значення
type	string	Критичний параметр. Визначає геометричну фігуру та її роль. Можливі значення: action (проста дія, прямокутник), question

		(умова, ромб), header (заголовок/початок алгоритму), end (термінатор), insertion (вставка підпрограми, макрос), case (елемент множинного вибору), address (перехід до іншого силуету). ⁵
content	string	Основний текстовий вміст ікони. Залежно від конфігурації віджета, може рендеритися як plain текст, markdown або html. ⁵ Саме сюди LLM повинна вставляти псевдокод або бізнес-логіку.
one (або next)	string	Ідентифікатор (ID) вузла, який є наступним за логікою виконання по "шампуру" (основному вертикальному шляху). ⁵
two (або left, zero)	string	Ідентифікатор вузла для альтернативної гілки (зазвичай це вихід "Ні" для ікони типу question або бічні гілки для case). ⁵
secondary	string	Додаткове текстове поле для специфічних ікон (наприклад, ікони "Полиця" або "Цикл"). ⁵
branchId	integer	Ідентифікатор гілки "Силуету". Дозволяє алгоритмам розбивати складні системи на паралельні логічні колонки. ⁵

style	string	Локальне перевизначення CSS/візуальних параметрів виключно для цього вузла (колір фону, товщина ліній).
-------	--------	---

✓ Переваги такої структури для генерації через AI (LLM):

1. **Мінімізація галуцинацій структури:** Моделям штучного інтелекту складно утримувати контекст глибоко вкладених дерев (як у AST). Згенерована дужка, що закривається не в тому місці, руйнує весь JSON. Плоский словник `items` дозволяє LLM генерувати об'єкти ітеративно, просто посилаючись на раніше згенеровані текстові ID.
2. **Відокремлення рендерингу від топології:** Зверніть увагу, що у форматі `drakonwidget` майже відсутні координати X та Y (хоча вони можуть опціонально кешуватися). Віджет розраховує геометрію та маршрутизацію ліній (Manhattan routing) динамічно в процесі рендерингу на основі зв'язків `one` та `two`.⁶ LLM не потрібно вираховувати пікселі чи уникати перетинів ліній — їй достатньо побудувати правильний логічний граф, а віджет сам забезпечить бездоганну математичну ергономіку (правило "Шампура" і заборону перетинів).¹

⚠ **Потреба у жорстких схемах (Transition Schemas):** Як зазначається в аудиті екосистеми¹², відсутність суворих API-контрактів та типізації призводить до критичних помилок. Оскільки оригінальний формат Міткіна не має офіційної JSON Schema (відсутні обов'язкові декларації типів), при інтеграції з LLM необхідно буде написати жорстку валідаційну схему (наприклад, за допомогою бібліотеки `Zod` у `TypeScript`). Ця схема повинна перевіряти, що поле `type` містить лише дозволені значення (`enum`), а ID, на які посилаються поля `one` та `two`, дійсно існують у словнику `items`.

5. DrakonWidget Deep Dive

Серед усіх компонентів екосистеми бібліотека `drakonwidget.js` (поточна версія 1.4.7⁵) є центральним елементом для інтеграції. Її архітектура та API визначають, наскільки легко можна буде побудувати сучасний користувацький інтерфейс.

5.1 Viewer чи Editor? Обидва режими

Віджет не є монолітною сутністю з єдиною поведінкою; він є гібридним рушієм, який може перемикатися між режимами переглядача та повноцінного редактора за допомогою конфігурації.

- **Режим Viewer (Переглядач):** Активується шляхом передачі властивості `canSelect: false` в об'єкті конфігурації під час ініціалізації. У цьому стані віджет блокує будь-яке виділення елементів, їх переміщення, зміну розмірів та редагування тексту. Він

перетворюється на інтерактивне полотно для читання (з підтримкою масштабування та панорамування).⁵

- **Режим Editor (Редактор):** Це поведінка за замовчуванням (`canSelect: true`). Користувач отримує можливість виділяти вузли (виділення супроводжується візуальними маркерами, так званими "candy markers" із зеленим заливом `candyFill` за замовчуванням⁵). Відкривається можливість змінювати ширину ікон (`allowResize: true`), а також користуватися вбудованою системою буфера обміну (`copySelection`, `cutSelection`, видалення).⁵ При цьому віджет відстежує стан через власну систему скасування/повтору (`undo/redo`), де метод `redo()` повертає JavaScript Promise.⁵

5.2 Технологія рендерингу: DOM проти Canvas

Віджет має надзвичайно цікаву стратегію рендерингу, керовану параметром `canvasIcons` (`boolean`, за замовчуванням `false`).

- **Гібридний рендеринг (`canvasIcons: false`):** Манхеттенська маршрутизація ліній, тіні та з'єднувальні вузли малюються на елементі HTML5 `<canvas>`. Проте контент самих ікон (насамперед текст) рендериться як абсолютно спозиційовані HTML DOM-елементи поверх полотна.⁵ Це критично важливе архітектурне рішення. Воно дозволяє браузеру нативно обробляти виділення тексту курсором, підтримувати високу чіткість шрифтів (`subpixel rendering`) та нативно рендерити HTML-теги або Markdown-розмітку всередині блоків.⁵
- **Повний Canvas-рендеринг (`canvasIcons: true`):** Віджет примусово малює весь текст та графіку безпосередньо на `CanvasContext`. Це знижує інтерактивність контенту, але є обов'язковою умовою для коректної роботи методу `exportCanvas()`, який дозволяє експортувати схему як єдиний PNG/BASE64-артефакт для збереження чи друку.⁵

5.3 Життєвий цикл інтеграції та API колбеків

Віджет написаний в імперативному стилі "vanilla JS", тому його не можна просто вставити як React-компонент `<DrakonWidget />`. Життєвий цикл інтеграції вимагає ручного управління⁵:

1. **Ініціалізація:** Виклик фабричної функції `var drakon = createDrakonWidget()`.
2. **Конфігурація Делегатів (Callbacks):** ⚠ Найважливіший технічний факт полягає в тому, що **віджет не має вбудованого інтерфейсу для введення тексту (`textarea`) або контекстних меню**. Він повністю покладається на "хост-застосунок" для реалізації цих UI-компонентів.⁵ Розробник зобов'язаний передати у конфігурацію функції-колбеки:
 - `startEditContent(item, isReadOnly)`: Коли користувач подвійним кліком намагається змінити текст вузла, віджет викликає цю функцію. Хост-застосунок (наш React-код) повинен показати модальне вікно або in-line редактор, отримати новий текст, і після підтвердження викликати API віджета

`drakon.setContent(itemId, newText)`, щоб зберегти зміни. Це дає феноменальну гнучкість: ми можемо використовувати потужні редактори на кшталт `Slate.js` або `TipTap` всередині ікон **DRAKON**.

- `showContextMenu(left, top, items)`: Аналогічно, при правому кліку віджет не малює меню сам, а передає масив доступних дій та координати у хост-застосунок, який має відрендерити власний UI-компонент (наприклад, за допомогою бібліотеки `Floating UI`).

3. **Рендеринг**: Виклик `drakon.render(width, height, config)` повертає DOM-елемент, який необхідно вручну прикріпити (`appendChild`) до контейнера на сторінці.⁵ Якщо контейнер змінює розмір (наприклад, при зміні розміру вікна), потрібно викликати `render()` знову, а потім метод `drakon.redraw()` для оновлення полотна.⁵

5.4 Синхронізація стану та `EditSender`

Для збереження змін у зовнішньому середовищі віджет використовує патерн обміну повідомленнями через об'єкт `EditSender`.⁵ При виклику методу завантаження діаграми `drakon.setDiagram(diagramId, diagram, sender)`, ми передаємо екземпляр цього об'єкта.

Будь-які структурні зміни, зроблені користувачем (додавання ікони, зміна зв'язку, видалення), не просто мутують внутрішній стан віджета. Віджет упаковує ці зміни у вигляді "дельт" (операцій типу `update`, `insert`, `delete`) і викликає метод `pushEdit(edit)` об'єкта `EditSender`.⁵  Це ідеальна архітектура для сучасних `React/Redux` систем. Ми можемо перехоплювати ці дельти і зберігати їх у глобальному сховищі стану (`Zustand/Redux`). Крім того, така дельто-орієнтована архітектура відкриває прямий шлях до реалізації спільного редагування у реальному часі (`Multiplayer`) через інтеграцію з `CRDT`-бібліотеками (наприклад, `Yjs` або `Automerge`).

6. Integration Assessment

Оцінка інтеграційного потенціалу екосистеми Степана Міткіна для розробки сучасного AI-орієнтованого вебзастосунку на стеку `TypeScript/React/Node.js` вимагає суворого сортування компонентів на ті, що готові до використання, ті, що потребують адаптації, і ті, які є технічним "баластом".

Що можна вбудувати без переписування (**Reuse**)

Бібліотеку `drakonwidget.js` необхідно взяти "як є" (без внесення змін у її вихідний код).

В основі мови **DRAKON** лежать топологічні інваріанти: заборона на перетин ліній зв'язку та використання суто ортогональної (Манхеттенської) геометрії.¹ Розробка власного рушія компонування графів, який би обчислював маршрути для обходу блоків лініями (алгоритми типу A^* або Дейкстри на ґратках) та динамічно вирівнював блоки на центральній осі ("шампурі"), потребуватиме багатьох місяців інженерної роботи.⁷ Віджет Міткіна ідеально абстрагує цю неймовірну математичну складність. Він детерміновано розраховує координати на основі простих логічних зв'язків `JSON`. Оскільки він не має

NPM-залежностей, його можна безпечно імпортувати як статичний асет.

Що потребує адаптації (Adapt)

Інтеграція віджета у декларативний життєвий цикл (React/Vue Wrapper).

Імперативна природа drakonwidget вступає в конфлікт з декларативним підходом React. Обсяг робіт середньої складності включає розробку спеціалізованого компонента-адаптера (наприклад, `<DrakonCanvas />`). Цей компонент повинен:

1. Використовувати `useRef` для отримання доступу до контейнера.
2. Ініціалізувати `createDrakonWidget()` всередині хука `useEffect` (або `onMounted` у Vue).
3. Використовувати `ResizeObserver` для відстеження розмірів контейнера та автоматичного виклику `drakon.render()` і `drakon.redraw()`.
4. Інкапсулювати логіку `EditSender`, перетворюючи імперативні дельти віджета на декларативні `dispatch`-дії (Actions) у глобальному стані застосунку.

Також адаптації потребують модулі кодогенерації (перетворення графа в Abstract Syntax Tree). Концепції та евристички обходу графа можна запозичити з папки `generator/` репозиторію `drakon.tech`⁹, але сам код доведеться переписати на сучасний TypeScript для забезпечення суворої типізації та модульності.

Що краще написати заново (Rewrite)

Бекенд-інфраструктуру та систему зберігання даних (DrakonHub Backend). Існуючий хмарний бекенд проєкту DrakonHub є яскравим прикладом архітектури, де legacy-концепції стають на заваді. Використання in-memory бази даних Tarantool³, у поєднанні з автогенерованим з Tcl кодом мовою Lua³, створює середовище з "bus factor", близьким до одиниці. Підтримка, масштабування (горизонтальний шардинг), сек'юриті-аудит та інтеграція сучасних протоколів (GraphQL, WebSockets) у такий стек будуть економічно невиправданими.³ Рішення: серверна частина повинна бути написана з нуля на Node.js (NestJS) або Go, з використанням надійної реляційної бази даних (наприклад, PostgreSQL). Роль бекенда має зводитися до валідації JSON-схем, авторизації, забезпечення управління версіями графів та оркестрації взаємодії з API великих мовних моделей (OpenAI, Anthropic).

Інтерфейс користувача (UI) для редагування.

Оскільки віджет делегує ці функції, всі модальні вікна, панелі інструментів (Toolbars) та контекстні меню необхідно розробити з нуля, використовуючи сучасні бібліотеки компонентів (MUI, Tailwind UI чи Radix).

Приклади успішного вбудовування

Документація проєкту **Motia**⁷ описує архітектурний план модернізації `drakonwidget.js` для використання у сучасному Single Page Application. Проєкт базується на концепції конвеєра "code-to-diagram", де аналіз абстрактного синтаксичного дерева (AST) наявного TypeScript

коду використовується для програмної генерації JSON-структури, яка потім "згодовується" віджету для візуалізації. Це підтверджує, що відокремлення логіки рендерингу віджета від управління даними є не лише можливим, а й практично перевіреним архітектурним патерном.

7. Build vs Reuse vs Adapt Decision Matrix

Ця матриця прийняття рішень підсумовує архітектурну стратегію роботи з компонентами екосистеми Міткіна.

Компонент	Рекомендація	Обґрунтування	Складність інтеграції	Пріоритет
Алгоритм макетування та топології (Manhattan Routing)	Reuse (drakonwidget.js)	Віджет бездоганно вирішує надскладну математичну задачу автоматичного позиціонування ікон (за правилом "Шампура") та обходу ліній без перетинів. ¹ Переписування з нуля вимагатиме місяців R&D.	Низька (залежить лише від API віджета)	Найвищий
DOM-інтеграція та Управління станом	Adapt (React/Vue Wrapper)	Віджет використовує застарілу імперативну парадигму. Його потрібно обгорнути у декларативний компонент, а	Середня (вимагає розуміння lifecycle хуків)	Високий

		об'єкт EditSender ⁵ під'єднати до сучасних State Менеджерів (наприклад, Zustand).		
UI-елементи (Контекстне меню, Текстовий редактор)	Build (з нуля)	Віджет свідомо не містить цих елементів "з коробки". ⁵ Вони мають бути реалізовані як сучасні React-компоненти та прив'язані до колбеків startEditContent / showContextMenu. ⁵	Низька	Високий
Бекенд (Сервер, Сховище БД, API)	Build (з нуля)	Legacy-стек DrakonHub (Tarantool, автозгенерований Lua-код через Tcl ³) є технологічним тягарем. Необхідно створити новий бекенд на Node.js/Go + PostgreSQL, який оброблятиме JSON-контрак	Висока (архітектура системи з нуля)	Високий

		ти. ¹²		
AI-Генератор (Text-to-Diagram)	Build (з нуля)	В екосистемі відсутні інтеграції для генерації схем штучним інтелектом (є лише експорт псевдокоду ⁸). Потрібно розробити систему Prompt Engineering, яка змусить LLM повертати правильну JSON-схему (adjacency list).	Середня	Середній
Кодогенератор (Diagram-to-Code / AST)	Adapt (логіка з drakon.tech)	Концепції та евристички обходу візуального графа для генерації Abstract Syntax Tree (AST) можна запозичити з папки generator/ в drakon.tech ⁹ , але переписати алгоритми на TypeScript.	Висока	Низький (на Етап 2)

8. Artifacts to Download

Список ключових цифрових артефактів з репозиторіїв Степана Міткіна, які необхідно локально зберегти для розробки та аудиту.

Файл / Репозиторій	Навіщо	Де знайти
Бібліотека drakonwidget.js	Основне ядро рендерингу (vanilla JS). Оскільки бібліотека не розповсюджується через NPM-пакети, мініфікований або сирцевий файл потрібно інтегрувати в проєкт як статичний ассет.	Репозиторій drakonwidget, папка libs/ або js/ (github.com/stepan-mitkin/drakonwidget). ⁵
Документація drakonwidget (Reference)	Містить повний опис усіх методів API, типів ікон (action, question тощо) та об'єкта theme (для кастомізації кольорів iconBack, candyFill, товщини ліній).	У README.md репозиторію drakonwidget та за посиланням app.drakon.tech/ide/doc/drakon_widget_10/1. ⁵
Тестові JSON-файли (схеми)	Еталонні зразки плоских словників items необхідні для аналізу структури графа, написання TypeScript-інтерфейсів (d.ts) та Zod-схем валідації.	Демонстраційна сторінка віджета (функція "Export JSON") ⁵ або репозиторій drakonhub/apptest.
Директорія generator/	Кодова база містить референсні алгоритми трансляції топології ДРАКОН (як правильно обробляти цикли та "Силуети") в AST-дерева.	Репозиторій drakon.tech, гілка master (github.com/stepan-mitkin/drakon.tech/tree/master/generator). ⁹

9. Open Technical Questions (Unresolved Questions)

Незважаючи на вичерпність аналізу кодових баз, залишається кілька критичних архітектурних питань, які потребують підтвердження шляхом створення інженерних

прототипів (Proof of Concept):

1. **? Толерантність до помилок у JSON (Hallucination Resilience):** Як саме поведе себе drakonwidget, якщо LLM згенерує пошкоджену логічну структуру? Наприклад, якщо поле next посилається на id, якого не існує в об'єкті items, чи виникне неперехоплювана помилка JavaScript (fatal error), яка "вб'є" рендеринг всієї сторінки? Якщо віджет не має вбудованої системи відмовостійкості, на стороні бекенда або клієнта знадобиться впровадження жорсткого шару попередньої валідації графа (Validation middleware) перед передачею даних у метод setData().
2. **? Продуктивність при масштабуванні графа:** Як гібридний рендеринг (canvasIcons: false), де текст рендериться тисячами DOM-елементів поверх Canvas⁵, вплине на продуктивність сучасних браузерів при завантаженні масштабних діаграм (наприклад, понад 1000 вузлів)? Чи реалізовані у віджеті механізми Culling (відмальовування лише тих вузлів, що знаходяться у видимому вікні — Virtual Viewport)?
3. **? Динамічна зміна тематики (Theming):** Документація згадує методи setDiagramStyle та patchDiagramStyle.⁵ Чи можливо за допомогою них реалізувати миттєве перемикання між "Світлою" (Light) та "Темною" (Dark) темами застосунку без необхідності повного перемонтування (unmount/remount) інстансу віджета та втрати локального стану зуму/панорамування?
4. **? CRDT Сумісність:** Інтерфейс EditSender генерує дискретні операції (дельти) типу update, insert, delete.⁵ Чи є структура цих повідомлень достатньо атомарною та детермінованою, щоб її можна було безпосередньо інтегрувати зі структурами даних CRDT (Conflict-free Replicated Data Types) бібліотек Yjs або Automerge для реалізації повноцінного багатокористувацького редагування в реальному часі?

10. Curated Bibliography

Для подальшого R&D процесу сформовано курований перелік ключових джерел з екосистеми:

1. **DrakonWidget Source & Docs** (stepan-mitkin/drakonwidget): Ключовий репозиторій для інтеграції візуального рушія. Ліцензія Unlicense..⁵
2. **Drakon.Tech IDE** (stepan-mitkin/drakon.tech): Репозиторій містить логіку трансляції схем DRAGON у виконуваний код (Clojure, JS) та абстрактні дерева синтаксису (AST)..⁹
3. **Drakon.Tech Desktop Wrapper** (stepan-mitkin/drakon.tech.desktop): Містить актуальні напрацювання щодо конвертації візуальних графів у промпти (псевдокод) для великих мовних моделей (ChatGPT, Gemini)..⁸
4. **DrakonHub Web Application** (stepan-mitkin/drakonhub): Зразок повної архітектури хмарного редактора, хоча його Tarantool-бекенд слугує радше антипатерном для сучасних SPA..³
5. **DRAKON Editor Fundamental** (stepan-mitkin/drakon_editor): Фундаментальний

Tcl-проект. Важливий виключно для розуміння форматів .drn (SQLite) та історичного контексту кодогенерації.²

6. *Внутрішні документи (Google Workspace):* "Покращення DRAKON Viewer для Motia"⁷, "Дослідження мови візуального програмування DRAKON"¹, "Технічний аудит та план розвитку AI-платформи".¹² Аналіз цих документів надав критичний контекст щодо застосування JSON-схем, семантичних мапперів AST та принципів топології (шампур/силует).

Executive Summary (Фінальне Резюме)

Аналіз екосистеми інструментів Степана Міткіна доводить, що її архітектура є сумішшю геніальних візуальних інновацій та застарілих серверних технологій. Для розробки сучасного AI-орієнтованого вебзастосунку **єдиною правильним архітектурним рішенням є вилучення фронтенд-ядра — бібліотеки drakonwidget.js — та відмова від усіх інших компонентів екосистеми (бекендів, форматів зберігання, десктопних застосунків)**. Цей віджет є зрілим, незалежним модулем на vanilla JavaScript, що віртуозно вирішує надскладну проблему Манхеттенського обходу ліній та підтримки топології "Шампура". Крім того, віджет приймає дані у вигляді плоского JSON-словника, що є ідеальним форматом для безпомилкової генерації графів великими мовними моделями. Всі серверні компоненти (написані на автозгенерованому Lua для бази Tarantool) та UI-елементи слід спроектувати та розробити з нуля на сучасному стеку (Node.js/Go, React/Vue), інтегрувавши drakonwidget.js як підпорядкований компонент візуалізації.

Structured Findings

-  **Клієнтський Рендеринг (Готовність до Reuse):** Бібліотека drakonwidget.js функціонує без залежностей, підтримує як режим перегляду, так і режим редагування (регулюється прапорцем canSelect), і забезпечує автоматичну математичну маршрутизацію ліній без перетинів.⁵
-  **AI-оптимізований формат даних:** Логіка зберігається не у глибоких деревах і не у бінарних базах SQLite (як у drakon_editor), а у плоскому JSON-словнику (items), де зв'язки графа описуються лінійними посиланнями на ID (one, two, next).⁵ Такий формат мінімізує структурні галюцинації LLM.
-  **Безперешкодне ліцензування:** Усі необхідні для розробки репозиторії випущені під ліцензією Unlicense або у Public Domain, що усуває будь-які копірайт-перешкоди для комерційної та пропрієтарної розробки.³
-  **Делегування UI:** Віджет навмисно не має власного інтерфейсу для введення тексту чи відображення меню. Розробник має впроваджувати власні React/HTML компоненти, викликаючи їх через делеговані колбеки startEditContent та showContextMenu.⁵
-  **Управління мутаціями (State Management):** Замість надання цілісного нового

стану при кожній зміні, віджет використовує об'єкт EditSender для генерації гранулярних "дельт" (операцій update/insert/delete).⁵ Цей патерн вимагатиме створення прошарку адаптації для глобальних State Менеджерів на зразок Redux.

- **⚠ Бекенд як антипатерн:** Серверна архітектура DrakonHub (in-memory база Tarantool з бізнес-логікою на Lua, згенерованою з Tcl³) є маргінальним та застарілим стеком, який недоцільно адаптувати чи розширювати для сучасного проєкту.

Unresolved Questions

- Як відреагує рушій віджета на недійсні топологічні структури у JSON (наприклад, посилання на неіснуючі ID, згенеровані через галюцинації LLM)? Чи потрібен додатковий шар пре-валідації перед викликом setData()?
- Чи забезпечує гібридний рендеринг (коли текст малюється через DOM-елементи поверх Canvas) прийнятний рівень продуктивності при відображенні алгоритмічних масивів понад 1000 іконок?
- Наскільки атомарними є дельти, які генерує EditSender, і чи їх достатньо для прямої конвертації в операції Conflict-free Replicated Data Types (CRDT) для реалізації багатокористувацького режиму?
- Чи підтримує API віджета плавну динамічну зміну візуальної теми (наприклад, з Light на Dark mode) без необхідності руйнувати (unmount) і перестворювати інстанс бібліотеки з втратою контексту масштабування?

Recommended Next Research Tasks

(Завдання для розширення інженерної бази у Промпті 3 або 4)

1. **Проектування React/Vue Адаптера:** Написати детальне технічне завдання (Technical Specification) для компонента-обгортки, який інкапсулюватиме імперативний життєвий цикл drakonwidget.js (реєстрація колбеків startEditContent, обробка ResizeObserver та синхронізація EditSender з декларативним станом Redux/Zustand).
2. **Розробка Zod Validation Schema:** Створити строгу типізовану схему (TypeScript Type / Zod Schema) для Canonical Data Model, яка базуватиметься на словнику items. Ця схема буде використовуватися як валідаційний бар'єр між відповіддю LLM та фронтендом.
3. **Prompt Engineering Концепція (Text-to-Diagram):** Розробити базовий системний промпт (System Prompt), який навчатиме великі мовні моделі (наприклад, GPT-4o) правилам побудови JSON-словників DRAKON: як призначати ідентифікатори, формувати алгоритмічний "Шампур" без перетинів і дотримуватися синтаксису розгалужень (question з полями one та two).

Джерела

1. Дослідження мови візуального програмування DRAKON,

<https://drive.google.com/open?id=1eO4walOtgKGpgsM9EW3wTd6S7TLRvOnGeOZr8DZ3NoM>

2. drakon_editor/README.md at master · stepan-mitkin/drakon_editor ..., доступ отримано травня 9, 2026,
https://github.com/stepan-mitkin/drakon_editor/blob/master/README.md
3. stepan-mitkin/drakonhub - GitHub, доступ отримано травня 9, 2026,
<https://github.com/stepan-mitkin/drakonhub>
4. stepan-mitkin/drakonhub_desktop: The desktop version of the DrakonHub diagram editor, доступ отримано травня 9, 2026,
https://github.com/stepan-mitkin/drakonhub_desktop
5. stepan-mitkin/drakonwidget: A JavaScript widget for viewing and editing drakon flowcharts - GitHub, доступ отримано травня 9, 2026,
<https://github.com/stepan-mitkin/drakonwidget>
6. Дослідження мови візуального програмування DRAKON.md
7. Покращення DRAKON Viewer для Motia,
https://drive.google.com/open?id=1_Zg6Tp9adKH1yWOqBFS8t7kcKLVW71LkjJ3viz-TEEI
8. stepan-mitkin/drakon.tech.desktop: A flowchart editor that generates prompts for AI apps and source code in Clojure and JavaScript - GitHub, доступ отримано травня 9, 2026, <https://github.com/stepan-mitkin/drakon.tech.desktop>
9. stepan-mitkin/drakon.tech - GitHub, доступ отримано травня 9, 2026,
<https://github.com/stepan-mitkin/drakon.tech>
10. drakon.tech/LICENSE at master · stepan-mitkin/drakon.tech · GitHub, доступ отримано травня 9, 2026,
<https://github.com/stepan-mitkin/drakon.tech/blob/master/LICENSE>
11. stepan-mitkin/drakon_editor: DRAKON Editor - GitHub, доступ отримано травня 9, 2026, https://github.com/stepan-mitkin/drakon_editor
12. Технічний аудит та план розвитку AI-платформи,
https://drive.google.com/open?id=1c8qOV6BHw1uQtflSE8HHwPNCQuHQOYM_B6w8miZNab8